

Heaps, Intervals, and Greedy Proofs

Heaps maintain an extreme under updates, interval problems depend on endpoint semantics, and greedy algorithms need a proof that a local choice preserves an optimal solution. These tools often combine in scheduling and streaming problems, but each solves a different part of the reasoning.

For: Candidates who can use a priority queue but want to know when it is the right structure and how to justify a greedy choice instead of trusting that it seems locally best.

Suggested pace: 2-3 weeks of focused variants | Sorting and basic DSA understood

What you will be able to do

- Maintain top-k or next-event state without sorting all data repeatedly.
- Define interval overlap and tie handling before implementation.
- Use exchange or dominance arguments to justify greedy decisions.
- Recognize when weights or extra constraints require DP or another method.

Build the right mental model

A heap exposes one useful extreme

A min-heap gives the smallest element efficiently, but does not keep every element in sorted order for arbitrary queries. Top-k streaming commonly uses a size-k min-heap so its root is the weakest retained candidate. State what every heap entry means.

Endpoints are part of the contract

Half-open intervals $[start, end)$ allow a meeting ending at time t to share a room with one starting at t . Closed intervals may have different overlap rules. Equal start/end times and zero-length intervals need an explicit interpretation.

Greedy requires exchangeability

For maximum count of non-overlapping intervals, choosing the earliest finishing compatible interval leaves at least as much room as another first choice. Weighted profit changes the objective and can invalidate this argument, even though the input still consists of intervals.

Study with evidence.

Write a tiny counterexample to a tempting incorrect rule, such as choosing the shortest or highest-profit interval. Then state the proof for the chosen rule. Separate the algorithmic choice from the heap implementation that makes it efficient.

Sessions 1-3: Priority queue operations

Practice insertion, extraction, bounded top-k, and merging sorted streams. Decide min-heap versus max-heap and include stable tie-breakers when values alone do not uniquely order entries.

Deliverable and check: A trace showing the invariant after every update and $O(\log k)$ versus $O(\log n)$ costs.

Sessions 4-6: Interval contracts

Solve merging, overlap detection, room allocation, and removal-count variants. Sort on the field needed by the proof, not automatically by start. Test touching endpoints and multiple simultaneous events.

Deliverable and check: A small suite that distinguishes half-open and closed interval behavior.

Sessions 7-10: Greedy correctness

Study earliest-finish scheduling, reachable-range expansion, and monotone choices. Explain an exchange or dominance argument. Compare with a weighted variant where the greedy rule fails.

Deliverable and check: One proof and one valid counterexample, both expressed in the problem's exact objective.

Sessions 11-14: Streaming and mixed practice

Combine heaps with offline sorting, query processing, or multiple streams. Remove expired or stale heap entries as required. Run a mixed problem without a topic hint and justify the retained state.

Deliverable and check: A tested solution that separates event eligibility, priority, and expiration clearly.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Allocate the minimum number of meeting rooms

Meetings use half-open time intervals. For $[0,30)$, $[5,10)$, and $[15,20)$, determine the peak rooms required and explain why the first meeting can overlap both shorter meetings without requiring three rooms.

A worked approach

1. Sort by start time and keep a min-heap of end times for active meetings. Before admitting a new meeting, remove every end time less than or equal to its start, because touching endpoints do not overlap under this contract.
2. At time 0, push 30. At time 5, no room is free, so push 10 and record a peak of two. At time 15, remove 10, retain 30, and push 20; the active count remains two.
3. Track the maximum heap size over the sweep. Explain that the heap models currently active meetings, not every meeting ever seen, and that removing expired entries is essential to this interpretation.

Expected result

The minimum room count is two: overlapping meetings require separate rooms, and the sweep reuses every available room. Sorting plus heap updates takes $O(n \log n)$ time and $O(n)$ worst-case auxiliary space.

Verify the result

- Do $[0,5)$ and $[5,10)$ reuse one room?
- Are all ended meetings removed before counting active demand?
- Does the result reflect the peak rather than only the final heap size?

Extend the exercise

Change the goal to selecting a maximum-profit compatible subset. Show a case where earliest finish loses to a more valuable interval, then formulate a weighted scheduling DP instead of reusing the room-count algorithm.

Make the learning stick.

Is the heap array sorted?

No. It satisfies a parent-child priority invariant sufficient to expose the root. Iterating its storage does not generally produce sorted order. Use extraction or sorting when ordered enumeration is required.

Why keep a min-heap for the largest k values?

The smallest retained value is the replacement threshold. A new value no larger than it cannot improve the retained set; a larger value replaces it. This bounds memory to k rather than storing the entire stream.

How do I know a greedy choice is valid?

Provide an exchange, dominance, or invariant argument showing it can be part of an optimum. Test variants and counterexamples. Local attractiveness is not a proof, especially when weights or dependencies change the objective.

Avoid these traps

- Confusing heap order with fully sorted order.
- Leaving interval endpoint semantics unspecified.
- Applying an unweighted greedy rule to weighted optimization.

Ready for the next step?

- Heap entries and their priority have explicit meanings.
- Overlap rules and ties are tested.
- Greedy choices have a correctness argument.
- A changed objective triggers a fresh analysis.

Reliable next reads

- [Python heapq](#)
- [MIT algorithms course](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.